What Sheaf Theory Actually Gets You

Three tasks, three concrete wins, no hand-waving

We built a sheaf-theoretic program analyzer. Here is exactly where the math pays off—and where it doesn't.

Task 1 **Zero-annotation bug finding** on 20 classes (div-by-zero . . . data races). *No annotations, no specs.*

Task 2 **Behavioral equivalence**: proof or localized counterexample for two implementations.

Task 3 **Spec checking**: refinement-type spec vs. function $\rightarrow$ yes/no with structured evidence.

For each: **what does the sheaf framing concretely add** over the best standard tool for the same job?

Standard (abstract interpretation): Compute global fixpoint over all paths. Cost $\mathcal{O}(|D|^k)$; requires widening, loses precision non-monotonically.

Sheaf approach: two decoupled phases.

1. **Local inference** (AI, symbolic exec, ML, tests—any method): assign a `TypeSection` to each site. Cost $\mathcal{O}(n)$.
2. **Gluing check**: per overlap, ask Z3 if $\text{avail}_i \wedge \neg\text{req}_j$ is SAT. Cost $\mathcal{O}(|\text{overlaps}|)$ local queries.

```
gap = And(available.predicate, Not(required.predicate))
is_bug = z3.Solver().check(gap) == sat        # LOCAL query
```

Win: no loop invariants needed. You check whether adjacent sites *agree*—a local property.

Key insight: checking $\rho_{U_i \cap U_j}$ is total is cheaper than constructing $s \in \Gamma(X, \mathcal{F})$.

47 warnings—how many are independent? Standard tools: unknown.

Sheaf answer: obstructions live in H^1 with algebraic structure. Build the **coboundary matrix** $\partial_0 \colon C^0 \to C^1$ over $\mathbb{F}_2$:

- Rows = overlaps, columns = sites; $(\partial_0)_{(i,j),k} = 1$ iff $k \in \{i, j\}$
- $\operatorname{rank} H^1 = \dim \ker \partial_1 - \dim \operatorname{im} \partial_0$ (Gaussian elimination over $\mathbb{F}_2$)

$\operatorname{rank} H^1$ = **minimum number of independent fixes**. Fix that many independent obstructions $\Rightarrow$ all 47 resolve.

Each obstruction carries a **gap predicate** $\varphi = \text{avail} \wedge \neg\text{req}$. Negate it $\to$ exact repair guard:

```
# Obstruction: gap = (y == 0)
repair_guard = Not(gap_predicate)   # => add ‘if y != 0:‘
```

Global bisimulation: k variables, domain d each. State space d^k, cost $\mathcal{O}(d^k \log d^k)$.

Sheaf decomposition into m sites, each touching k/m variables:

$$\underbrace{\mathcal{O}(m \cdot d^{k/m})}_{\text{local equiv.}} + \underbrace{\mathcal{O}(|\text{overlaps}| \cdot \text{SMT})}_{\text{gluing (poly)}} + \underbrace{\mathcal{O}(|\text{triples}|)}_{\text{cocycle}}$$

Since $m \cdot d^{k/m} \ll d^k$ for $m > 1$: **exponentially cheaper**.

When? Variables partition into nearly-independent groups—the common case (functions have local variables). The natural cover from code structure gives $m = \Theta(k/\log k)$ or better.

What you pay: cocycle check on triples ($\mathcal{O}(m^3)$ worst case, typically sparse). Dominated by local equivalence checks.

Local equivalences $\{\eta_i : \mathrm{Sem}_f(U_i) \xrightarrow{\sim} \mathrm{Sem}_g(U_i)\}$ glue to a global equivalence **iff** $H^1(\{U_i\}, \mathcal{I}so) = 0$.

- Standard modular checking: interfaces match $\Rightarrow$ equiv. **Sound but not complete**—rejects equivalent programs with non-matching interfaces.
- Sheaf: $H^1 = 0 \Leftrightarrow$ global equiv. **Sound and complete** for the cover.

When $H^1 \neq 0$, cocycle representatives localize failure to specific overlaps:

```
# Cocycle on triple (i,j,k): g_ij . g_jk == g_ik ?
lhs = Conjunction(transition[(i,j)].pred, transition[(j,k)].pred)
rhs = transition[(i,k)].pred
if not smt_equivalent(lhs, rhs):
    report(sites=(i,j,k), witness=counterexample)
```

Standard (Liquid Haskell, F*): one monolithic VC per function. n paths, k conjuncts $\to$ formula of size $\mathcal{O}(nk)$.

Sheaf: product cover $\{(p_i, c_j)\}$, one site per path $\times$ conjunct:

```
cover = { (path_i, conj_j)  for i in paths  for j in spec }
# Each VC is SMALL: one path, one conjunct
```

Payoffs:

1. **Parallel**: all nk VCs independent. Standard: one monolithic call.
2. **Incremental**: change $p_3 \to$ re-check row 3 only.
3. **Localized**: "conj 2 fails on path 4" vs "VC unsat."
4. **Proof transfer**: (p_1, c_j) proved $\wedge$ $p_1 \cap p_2 \neq \emptyset \Rightarrow$ skip (p_2, c_j).

Standard: "all VCs passed $\Rightarrow$ spec holds." But how do you know VCs *covered everything*?

Sheaf checking adds two checkable certificates:

1. H^0 (components of cover nerve): $H^0 = 1$ means the cover is connected—*no dead zones*.
2. H^1 (cycle rank of cover nerve): $H^1 = 0$ means proofs on overlapping sites are globally consistent—*no invisible interactions*.

Together: **all VCs pass** $\wedge$ $H^0 = 1$ $\wedge$ $H^1 = 0$ gives a *structured soundness argument*:

Coverage:	cover spans the function (H^0)
Consistency:	local proofs compose (H^1)
Correctness:	each local VC discharged (SMT)

No standard refinement checker offers an analogous meta-level completeness/consistency certificate.

Decompose f at a branch into A, B with overlap $A \cap B$. The exact sequence:

$$0 \to H^0(f) \to H^0(A) \oplus H^0(B) \to H^0(A \cap B) \xrightarrow{\delta} H^1(f) \to H^1(A) \oplus H^1(B) \to H^1(A \cap B)$$

Translation: H^0 = spec holds; H^1 = obstructions; δ = overlap failures $\to$ global obstructions.

What nothing else gives you:

1. **Compositional obstruction counting**: $\operatorname{rk} H^1(f) = \operatorname{rk} H^1(A) + \operatorname{rk} H^1(B) - \operatorname{rk} H^1(A \cap B) + \operatorname{rk} \operatorname{im}(\delta)$. Bug count of the whole from bug counts of the parts.
2. **Incremental**: modify $A \Rightarrow$ recompute $H^1(A)$ and $H^0(A \cap B)$, plug in. No re-analysis of B.
3. **Unification**: same sequence governs bugs (T1), equivalence (T2), and specs (T3).

Claim	Impl.	Status
Bug finding: 20 classes, zero annotations	✓	Shipped
Bug finding: gap predicate $\to$ repair guard	✓	New
Bug finding: rank H^1 via $\mathbb{F}_2$ Gaussian elim	✓	New
Bug finding: obstruction independence algebra	✓	New
Equivalence: Čech cocycle on triples	✓	Shipped
Equivalence: $H^1 = 0 \Leftrightarrow$ global equiv	✓	Shipped
Spec checking: product-cover VC factoring	✓	Shipped
Spec checking: proof transfer via overlap	✓	Shipped
Spec checking: $H^0 + H^1$ meta-certificate	✓	Shipped
Mayer-Vietoris connecting homomorphism	✓	New
Formal soundness proof (Lean/Coq)	×	Open
Benchmark vs Astrée / Liquid Haskell	×	Open

Bottom line: compositional analysis with provable cost, completeness, and failure structure—not vocabulary, but concrete algorithms with concrete advantages.